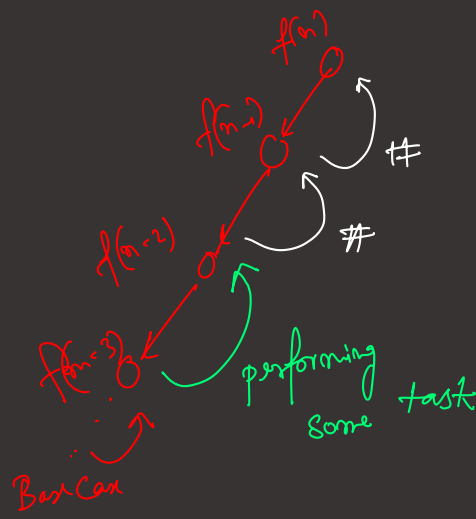
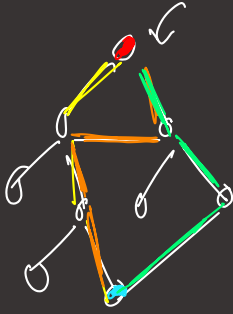


Backtracking

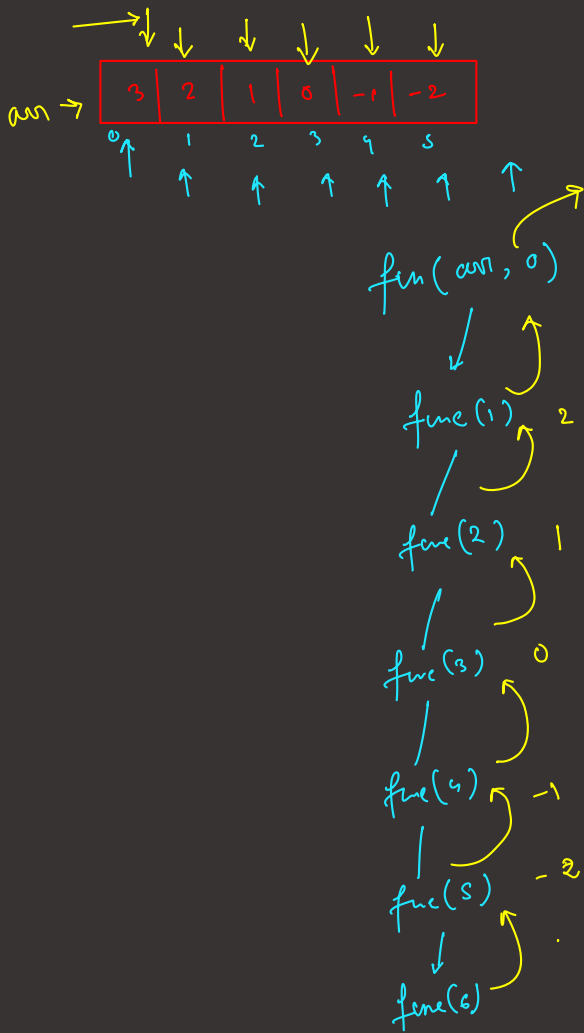


types - ~~1~~ All possible ways.

② Best way among all the solⁿ *

✓ any one way (it may not be best)

③ Decision → whether the solⁿ exist or not.



```

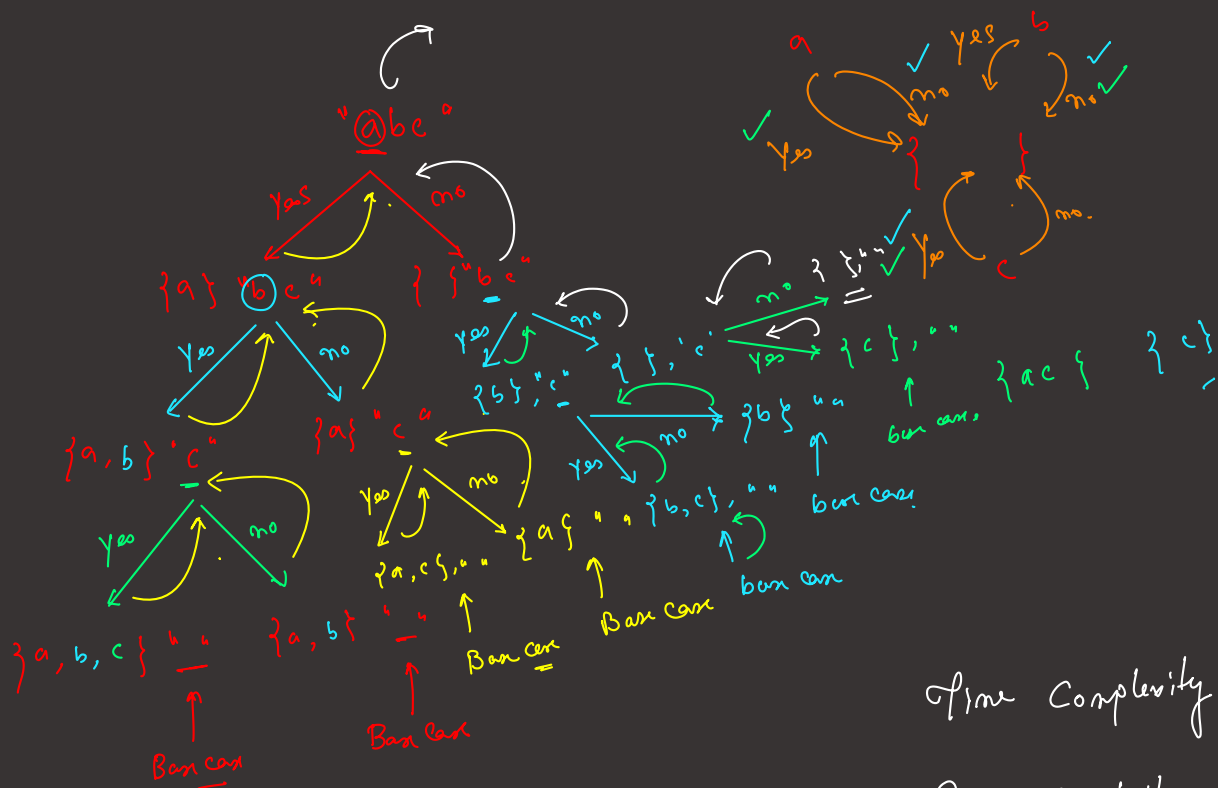
fun(arr, i) {
    if (i == arr.length) {
        return;
    }
    fun(arr, i+1);
    s.out(arr[i]);
}
    
```

o/p: -2 -1 0 1 2 3

Q Find and print all subsets of a given string

eg. str = "abc"

o/p: abc, ab, bc, ca, a, b, c, ""



Time Complexity: $O(2^n \times n)$

Space Complexity: $O(n)$

given string
findSubsets(set, ans, i)

if (i == set.length()) {

print(ans);

return;

}

// recursion

// yes

findSubsets(set, ans + set.charAt(i), i + 1);

// no

findSubsets(set, ans, i + 1);

}

f(abc, "", 0)

↓

f(abc, 'a', 1)

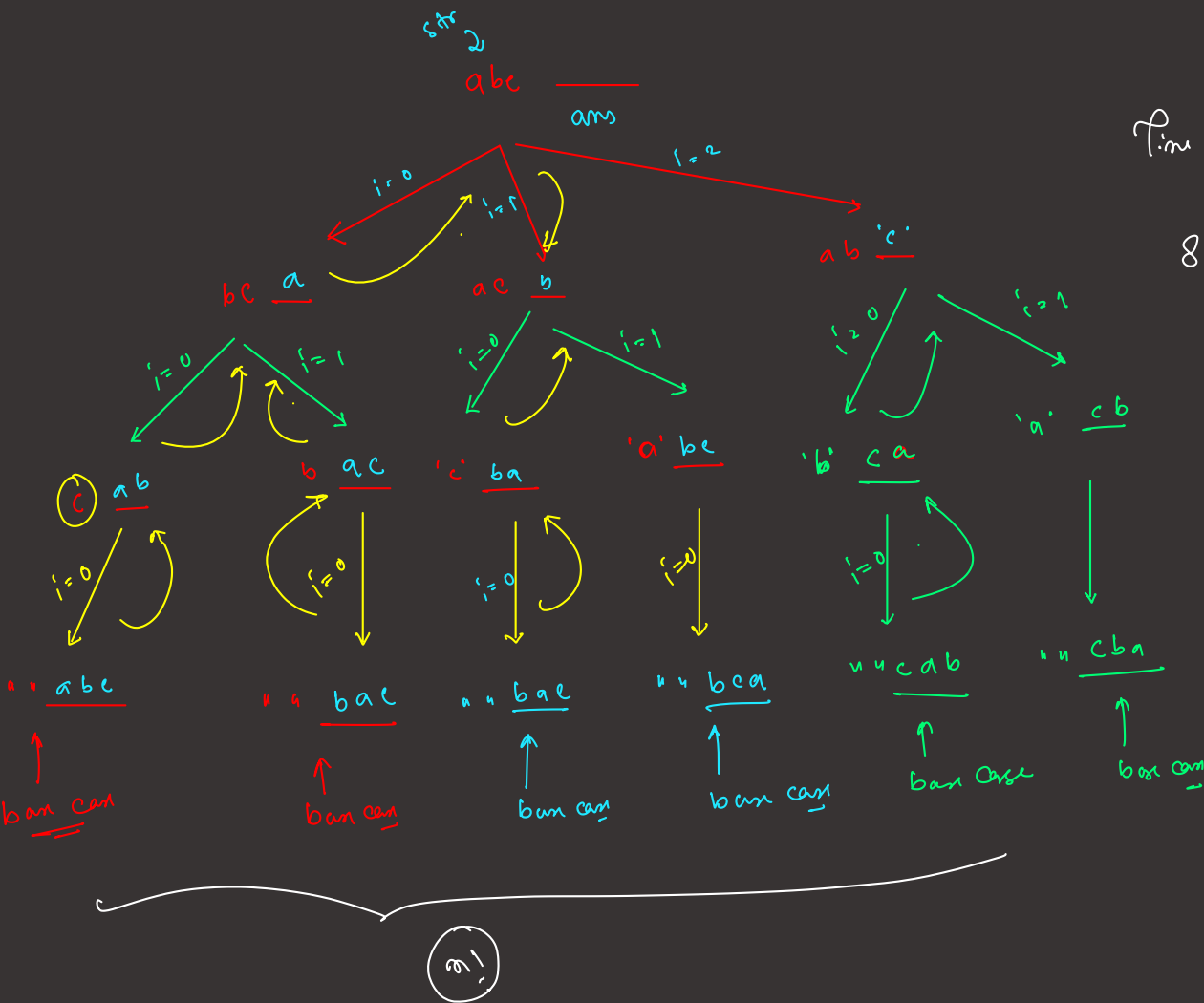
f(abc, 'b', 2)

Find Permutation

Print all the permutation of a string.

i/p: "abc"

o/p: "abc", "acb", "bac", "bca", "cab", "cba"



$\uparrow$ (b) c
 $\uparrow$ ac
 $\uparrow$ i
 abc @ 2 f
 abc + 2 f
 str, subbing (0, i)
 + str, subbing (i-1, i)

Space

$p(abc, "")$ $i=0, i=1, i=2$
generate?

o/p: abc, acb, bac, bca, ...

